# MP5: Simple Kernel Threads

Mohammed Sharique
UIN: 837004870
CSCE611: Operating System

03/29/2026

## Assigned Tasks

**Main:** Completed.
**Bonus 1:** Completed.
**Bonus 2:** Completed.
**Bonus 3:** Not Completed.

## System Design

The goal of this machine problem was to implement a preemptive Round Robin scheduler. The main idea was to keep FIFO queue management in the scheduler, then layer EOQ (end-of-quantum) preemption on top using a timer interrupt. In addition, I added safe interrupt acknowledgement methods for context switches to handle handle preemption during interrupt execution without sending duplicate end of interrupts to the PIC.

- I first implemented the basic Scheduler as a FIFO ready queue scheduler, maintains order of threads scheduled and performs the context switches between threads by dispatching to the thread next in queue.
- I then handled terminating thread functions by adding cleanup code in thread_shutdown module, that adds thread in the zombie queue. The resources held by threads in zombie queue is freed up when another thread resumes.
- After that, I added interrupt handling support required for scheduler driven preemption, including proper interaction between timer interrupts and thread context switching. I had to ensure that all the scheduler code is treated as a critical path and while the scheduler code is executing interrupts are disabled.
- Then I implemented EOQTimer which was used inside the Round Robin scheduler to track quantum progressc and trigger a preemption request when the configured quantum for a thread expires.
- To avoid penalizing threads after voluntary yields, I reset the quantum counter whenever a running thread yields before its time expires.
- I added explicit interrupt acknowledgement methods to make preemptive context switches safe inside timer interrupt handling. When end of quantum is reached, acknowledge_interrupt sends EOI immediately and marks the interrupt context as already acknowledged by setting the error code to a marker value. When control later returns through dispatch_interrupt(), the dispatcher checks that marker and skips sending EOI again, preventing duplicate acknowledgements to the PIC.

## Code Description

I modified the files scheduler.C/H, simple_timer.C/H, threads.C/H, interrupts.C and kernel.C (just to enable the round robin scheduler and terminating functions).
The code compiles by running make in the project directory.
Github: https://github.com/sharique-tamu/csce-611-machine-problems

**scheduler.C:sentinel_entry** : This function implements the idle execution loop used when no regular runnable thread is available. It continuously checks whether the global system scheduler pointer is valid and, if so, invokes yield. This guarantees that the kernel always has a safe execution path even when the ready queue is empty, instead of falling into an invalid dispatch state.

```
void Scheduler::sentinel_entry() {
  for (;;) {
    if (Thread::SYSTEM_SCHEDULER != nullptr) {
      Thread::SYSTEM_SCHEDULER->yield();
    }
  }
}
```

**scheduler.C:Scheduler (constructor)** : This constructor initializes the scheduler state for both normal scheduling and termination cleanup. It resets queue counters and indices, clears ready and zombie queues, allocates stack space for the sentinel thread, and creates the sentinel thread object. This setup ensures deterministic startup and prepares the scheduler to handle empty queue conditions safely.

```
Scheduler::Scheduler() {
  n_threads = 0;
  start = 0;
  running_thread = nullptr;
  sentinel_thread = nullptr;
  sentinel_stack = nullptr;
  z_threads = 0;

  for (int i = 0; i < MAX_THREADS; i++) {
    q[i] = nullptr;
    zombie_q[i] = nullptr;
  }

  sentinel_stack = new char[SENTINEL_STACK_SIZE];
  sentinel_thread = new Thread(&Scheduler::sentinel_entry, sentinel_stack,
                               SENTINEL_STACK_SIZE);

  Console::puts("Constructed Scheduler.\n");
}
```

**scheduler.C:yield** : This function performs the main dispatch operation of the FIFO scheduler. It first preserves interrupt state and disables interrupts to protect scheduling logic as a critical section. If the ready queue is empty, it dispatches sentinel (or returns immediately if sentinel is already running). Otherwise, it dequeues the next runnable thread from the circular queue, updates queue metadata, restores interrupts, and performs the context switch.

**scheduler.C:resume** : This function makes a thread runnable by appending it to the ready queue. Before enqueueing, it performs deferred cleanup of zombie threads by calling cleanup and deleting them. It ignores null and sentinel thread inputs, checks for queue overflow, and inserts valid threads at the queue tail while preserving FIFO ordering.

```cpp
void Scheduler::yield() {
  bool is_interrupts_enabled = Machine::interrupts_enabled();
  if (is_interrupts_enabled) {
    Machine::disable_interrupts();
  }

  if (n_threads == 0) {
    Thread *current_thread = Thread::CurrentThread();

    if (current_thread == sentinel_thread) {
      if (is_interrupts_enabled) {
        Machine::enable_interrupts();
      }
      return;
    }

    running_thread = sentinel_thread;

    if (is_interrupts_enabled) {
      Machine::enable_interrupts();
    }

    Thread::dispatch_to(sentinel_thread);
    return;
  }

  running_thread = q[start];
  q[start] = nullptr;
  start = (start + 1) % MAX_THREADS;
  n_threads--;

  if (is_interrupts_enabled) {
    Machine::enable_interrupts();
  }

  Thread::dispatch_to(running_thread);
}
```

**scheduler.C:add** : This function is a wrapper over resume and is used for adding newly created threads into scheduling. It centralizes all enqueue behavior in one place by directly delegating to resume.

**scheduler.C:terminate** : This function removes a thread from scheduling and transitions it to deferred destruction using the zombie queue. It protects scheduler data structures with interrupt disable/restore, rejects invalid and sentinel termination requests, and handles two cases: self-termination (enqueue current thread as zombie and yield immediately) and external termination (remove target from ready queue if present, compact queue, and enqueue as zombie). This prevents unsafe immediate destruction during active execution.

**scheduler.C:request_preemption** : This function is invoked by the EOQ timer path to enforce preemptive scheduling. It first verifies that a current thread exists and that preemption is meaningful (at least one other runnable thread exists). It then re-enqueues the current thread and calls yield so the scheduler can select the next thread according to FIFO order.

**scheduler.C:RRScheduler (constructor)** : This constructor extends the base scheduler with round-robin timing. It creates an EOQ timer with configured tick rate and quantum length, and registers that timer as IRQ0 handler. This links hardware timer interrupts directly to scheduler preemption decisions.

```cpp
void Scheduler::resume(Thread *_thread) {
  bool is_interrupts_enabled = Machine::interrupts_enabled();
  if (is_interrupts_enabled) {
    Machine::disable_interrupts();
  }

  while (z_threads) {
    Thread *del_thread = zombie_q[--z_threads];
    zombie_q[z_threads] = nullptr;
    del_thread->cleanup();
    delete del_thread;
    del_thread = nullptr;
  }
  if (_thread == nullptr) {
    if (is_interrupts_enabled) {
      Machine::enable_interrupts();
    }
    return;
  }

  if (_thread == sentinel_thread) {
    if (is_interrupts_enabled) {
      Machine::enable_interrupts();
    }
    return;
  }

  if (n_threads == MAX_THREADS) {
    if (is_interrupts_enabled) {
      Machine::enable_interrupts();
    }
    Console::puts("Max threads already in queue.\n");
    assert(false);
  }
  q[(start + n_threads++) % MAX_THREADS] = _thread;

  if (is_interrupts_enabled) {
    Machine::enable_interrupts();
  }
}
```

```cpp
void Scheduler::add(Thread *_thread) { resume(_thread); }
```

**scheduler.C:RRScheduler::yield** : This function overrides the base yield path to maintain round-robin fairness. Before delegating to Scheduler::yield, it resets EOQ quantum progress so the next dispatched thread starts with a full time slice.

**scheduler.C:RRScheduler::request_preemption** : This function currently delegates to the base scheduler preemption implementation. It preserves a policy-specific override point while reusing common preemption behavior (requeue current thread and yield).

**simple_timer.C:EOQTimer::EOQTimer** : This constructor initializes the end-of-quantum timer used by round-robin scheduling. It first initializes the base timer frequency through the parent SimpleTimer constructor, then stores the configured quantum length, clears the current quantum progress counter, and saves the scheduler pointer used for preemption callbacks. This sets up all state needed for timer-driven preemptive scheduling.

```cpp
void Scheduler::terminate(Thread *_thread) {
  bool is_interrupts_enabled = Machine::interrupts_enabled();
  if (is_interrupts_enabled) {
    Machine::disable_interrupts();
  }

  if (_thread == nullptr) {
    if (is_interrupts_enabled) {
      Machine::enable_interrupts();
    }
    return;
  }

  if (_thread == sentinel_thread) {
    if (is_interrupts_enabled) {
      Machine::enable_interrupts();
    }
    return;
  }

  if (z_threads == MAX_THREADS) {
    if (is_interrupts_enabled) {
      Machine::enable_interrupts();
    }
    Console::puts("Too many zombie threads.\n");
    assert(false);
  }

  Thread *current_thread = Thread::CurrentThread();

  if (_thread == current_thread) {
    zombie_q[z_threads++] = _thread;

    if (is_interrupts_enabled) {
      Machine::enable_interrupts();
    }

    yield();
    assert(false);
  }
```

**simple_timer.C:EOQTimer::reset_quantum** : This function resets the accumulated tick count for the current quantum window. It is used when a thread yields voluntarily so that the next scheduled thread starts with a fresh, full quantum instead of inheriting residual tick progress.

**simple_timer.C:EOQTimer::handle_interrupt** : This function handles timer interrupts for EOQ-based preemption. It first invokes the base timer handler to maintain general timer accounting, then increments quantum progress. If the quantum has not yet expired, it returns immediately. On quantum expiry, it resets the quantum counter, verifies that a scheduler is available, acknowledges the interrupt early, and requests preemption from the scheduler. This directly connects timer ticks to round-robin thread switching.

**thread.C:thread_shutdown** : This function is executed when a thread function returns. Instead of freeing resources directly in that execution context, it hands termination to the scheduler by requesting termination of the current thread. This allows the scheduler to place the thread in the zombie cleanup flow and avoid unsafe destruction during active execution. The final assertion ensures control does not continue past shutdown.

**thread.C:thread_start** : This function prepares a thread context for normal execution by ensuring interrupts are enabled before user thread work proceeds. Since scheduling and setup paths may run

```
  bool found = false;
  for (int i = 0; i < n_threads; i++) {
    int idx = (start + i) % MAX_THREADS;
    if (q[idx] == _thread) {
      found = true;
      for (int j = i; j < n_threads - 1; j++) {
        int from = (start + j + 1) % MAX_THREADS;
        int to = (start + j) % MAX_THREADS;
        q[to] = q[from];
      }
      int tail = (start + n_threads - 1) % MAX_THREADS;
      q[tail] = nullptr;
      n_threads--;
      break;
    }
  }

  if (!found && _thread != running_thread) {
    if (is_interrupts_enabled) {
      Machine::enable_interrupts();
    }
    return;
  }

  zombie_q[z_threads++] = _thread;

  if (is_interrupts_enabled) {
    Machine::enable_interrupts();
  }
}
```

```
void Scheduler::request_preemption() {
  Thread *current_thread = Thread::CurrentThread();
  if (current_thread == nullptr) {
    return;
  }

  if (n_threads == 0) {
    return;
  }

  resume(current_thread);
  yield();
}
```

```
RRScheduler::RRScheduler() : Scheduler() {
  eoq_timer = new EOQTimer(TIMER_HZ, QUANTUM_TICKS, this);
  InterruptHandler::register_handler(0, eoq_timer);
}
```

with interrupts disabled, this function restores expected runtime behavior for thread code without altering scheduler invariants.

```
void RRScheduler::yield() {
  eoq_timer->reset_quantum();
  Scheduler::yield();
}
```

```
void RRScheduler::request_preemption() { Scheduler::request_preemption(); }
```

```
EOQTimer::EOQTimer(int _hz, int _quantum_ticks, Scheduler *_scheduler)
    : SimpleTimer(_hz) {
  quantum_ticks = _quantum_ticks;
  ticks_since_quantum = 0;
  scheduler = _scheduler;
}
```

```
void EOQTimer::reset_quantum() { ticks_since_quantum = 0; }
```

```
void EOQTimer::handle_interrupt(REGS *_r) {
  SimpleTimer::handle_interrupt(_r);

  ticks_since_quantum++;
  if (ticks_since_quantum < quantum_ticks) {
    return;
  }

  ticks_since_quantum = 0;

  if (scheduler == nullptr) {
    return;
  }

  InterruptHandler::acknowledge_interrupt(_r);
  scheduler->request_preemption();
}
```

```
static void thread_shutdown() {
  /* This function should be called when the thread returns from the thread
     function. It terminates the thread by releasing memory and any other
     resources held by the thread. This is a bit complicated because the thread
     termination interacts with the scheduler.
   */
  Thread::SYSTEM_SCHEDULER->terminate(Thread::CurrentThread());
  assert(false);
}
```

```
static void thread_start() {
  /* This function is used to release the thread for execution in the ready
   * queue. */

  /* We need to add code, but it is probably nothing more than enabling
   * interrupts. */
  if (!Machine::interrupts_enabled()) {
    Machine::enable_interrupts();
  }
}
```

**thread.C:Thread::cleanup** : This function releases per-thread dynamically allocated resources after the thread has been safely retired. It frees the thread stack memory and the saved execution

cargo/context object. This is typically invoked from deferred cleanup paths (such as zombie processing) to avoid deleting active execution state.

```
void Thread::cleanup() {
  delete[] stack;
  delete cargo;
}
```

**interrupts.C:InterruptHandler::send_eoi** : This function sends the End-of-Interrupt (EOI) command to the PIC after an IRQ has been handled. If the interrupt originated from the slave PIC, it first acknowledges the slave controller at port 0xA0 and then acknowledges the master controller at port 0x20. For master-only IRQs, it sends EOI only to the master PIC. This preserves correct PIC acknowledgement order and ensures interrupts are re-armed properly.

```
void InterruptHandler::send_eoi(unsigned int _irq_no) {
  if (generated_by_slave_PIC(_irq_no)) {
    Machine::outportb(0xA0, 0x20);
  }

  Machine::outportb(0x20, 0x20);
}
```

**interrupts.C:InterruptHandler::acknowledge_interrupt** : This function performs early interrupt acknowledgement for preemptive interrupt paths. It computes the IRQ number from the interrupt frame, validates it against table bounds, and calls send_eoi immediately. After sending EOI, it marks the interrupt frame with EOI_ALREADY_SENT_MARK so the dispatcher can detect that acknowledgement was already completed and avoid sending a duplicate EOI later.

```
void InterruptHandler::acknowledge_interrupt(REGS *_r) {
  unsigned int irq_no = _r->int_no - IRQ_BASE;
  assert(irq_no < IRQ_TABLE_SIZE);

  send_eoi(irq_no);
  _r->err_code = EOI_ALREADY_SENT_MARK;
}
```

## Testing

For this machine problem I relied on the provided tests for validating the implementation. The test setup repeatedly exercised thread creation, context switching, and timer-driven preemption while observing serial output for scheduling order, timer progress, and system stability.

Initially, I encountered issues around preemptive switching during interrupt handling, especially in the interrupt acknowledgement path. This was resolved by introducing explicit acknowledgement logic that marks the interrupt context when EOI is sent early, so the dispatcher can avoid sending duplicate EOIs later. I also debugged scheduler edge cases related to terminating thread functions and empty ready queue behavior.

The final implementation correctly performs FIFO based scheduling and round robin preemption with EOQTimer, resets quantum state on voluntary yields, and safely handles terminating threads through zombie queue cleanup. Runtime output showed consistent interleaving of runnable threads, continued timer activity, and stable execution even when normal worker threads were exhausted, confirming correct scheduler and interrupt integration.